

Assignment No: - 7

Name :- Isha Ghorpade

Enrollment :- 22420020

Roll :- 381066

Problem Statement

Implement a better auto-complete algorithm using an N-gram language model.

The system should predict the next word based on the previous words using probability calculated from an N-gram model.

Objectives

- To understand the concept of language models in Natural Language Processing.
 - To study N-gram models for predicting next words.
 - To implement an auto-complete system using N-gram probability.
 - To analyze how context improves prediction accuracy.
-

Software and Hardware Requirements

Software

- Python 3.x
- Jupyter Notebook / Google Colab
- Libraries:
 - NLTK
 - NumPy
 - Collections

Hardware

- Computer or Laptop
 - Minimum 4 GB RAM
 - Internet connection (for dataset or NLTK download)
-

Theory

Auto-Complete System

An auto-complete system predicts the next word or phrase while the user is typing.

It is widely used in:

- Google Search
- Gmail Smart Compose
- Mobile keyboards
- Chat applications

Example:

Input:

I love machine

Prediction:

learning

N-Gram Language Model

An N-gram is a sequence of N words appearing together in text.

N	Name	Example
---	------	---------

1	Unigram	I
---	---------	---

2	Bigram	I love
---	--------	--------

3	Trigram	I love NLP
---	---------	------------

Example sentence:

I love natural language processing

Trigrams:

I love natural

love natural language

natural language processing

Probability in N-Gram Model

The probability of a word is calculated using previous words.

Example for Bigram Model

$P(\text{next word} \mid \text{previous word})$

Example:

$P(\text{love} \mid \text{I})$

For Trigram Model

$P(\text{word3} \mid \text{word1}, \text{word2})$

Example:

$P(\text{NLP} \mid \text{love}, \text{learning})$

Working of Auto-Complete Algorithm

Steps:

1. Take a large text corpus
 2. Perform preprocessing (lowercase, tokenization)
 3. Generate N-grams
 4. Calculate frequency counts
 5. Compute conditional probability
 6. Suggest the most probable next word
-

Algorithm

1. Import required libraries.
 2. Load text corpus.
 3. Tokenize the text into words.
 4. Generate N-grams.
 5. Count frequencies of each N-gram.
 6. Take user input sentence.
 7. Extract last N-1 words.
 8. Predict the next word using highest probability.
 9. Display suggested words.
-

Python Implementation

```
import nltk
from nltk.util import ngrams
from collections import defaultdict, Counter
import random
```

```
# Sample corpus
text = ""
```

machine learning is amazing
machine learning is powerful
machine learning helps computers learn
natural language processing is part of machine learning
""

Tokenization

```
tokens = nltk.word_tokenize(text.lower())
```

Create trigram model

```
n = 3
```

```
ngrams_list = list(ngrams(tokens, n))
```

Dictionary for storing predictions

```
model = defaultdict(list)
```

```
for w1, w2, w3 in ngrams_list:
```

```
    model[(w1, w2)].append(w3)
```

Convert lists to frequency counts

```
model_freq = {k: Counter(v) for k, v in model.items()}
```

Function for auto complete

```
def autocomplete(text):
```

```
    words = text.lower().split()
```

```
    if len(words) < 2:
```

```
        return "Enter at least two words"
```

```
    key = (words[-2], words[-1])
```

```
    if key in model_freq:
```

```
        predictions = model_freq[key].most_common(3)
```

```
        return [word for word, count in predictions]
```

```
    else:
```

```
        return "No prediction found"
```

Test the model

```
user_input = input("Enter a sentence: ")
```

```
print("Suggested words:", autocomplete(user_input))
```

Example Output

Input:

machine learning

Output:

['is', 'helps']

Input:

natural language

Output:

['processing']

Advantages of N-Gram Auto-Complete

- Simple and easy to implement
- Fast prediction
- Works well for small datasets
- Used as baseline models in NLP

Limitations

- Needs large datasets for accuracy
- Cannot understand deep context
- Limited prediction capability compared to deep learning models

Applications

- Search engines
- Mobile keyboards
- Email smart reply
- Chatbots
- Voice assistants

Conclusion

The N-gram model provides a simple yet effective approach for building an auto-complete system. By analyzing the probability of word sequences, the model predicts the most likely next word based on the previous context. Although more advanced models such as neural language models exist, N-gram models remain useful for understanding the fundamentals of language modeling.